```
import random

# Initialize the 3x3 board with numbers 1-9
board = [str(i) for i in range(1, 10)]
human = 'X'
computer = 'O'

def print_board():
    """Prints the current state of the board."""
    print("\n")
    print(f" {board[0]} | {board[1]} | {board[2]} ")
    print("---|---|---")
    print(f" {board[3]} | {board[4]} | {board[5]} ")
    print("---|---|---")
    print(f" {board[6]} | {board[7]} | {board[8]} ")
    print("\n")

def check_win(b, player):
    """Checks if the given player has won the game."""
    # Hardcoded matrix indices to prevent rendering syntax errors
    row1 = [0, 1, 2]
    row2 = [3, 4, 5]
    row3 = [6, 7, 8]
    col1 = [0, 3, 6]
    col2 = [1, 4, 7]
    col3 = [2, 5, 8]
    diag1 = [0, 4, 8]
    diag2 = [2, 4, 6]

    win_conditions = [row1, row2, row3, col1, col2, col3, diag1, diag2]
    return any(all(b[pos] == player for pos in condition) for condition in win_conditions)

def check_tie(b):
    """Checks if the game is a tie (all spots filled)."""
    return all(spot in ['X', 'O'] for spot in b)

def get_computer_move():
    """Calculates the best move for the computer."""
    # 1. Can the computer win in this move?
    for i in range(9):
        if board[i] not in ['X', 'O']:
            board_copy = board.copy()
            board_copy[i] = computer
            if check_win(board_copy, computer):
                return i

    # 2. Does the computer need to block the human?
    for i in range(9):
        if board[i] not in ['X', 'O']:
            board_copy = board.copy()
            board_copy[i] = human
            if check_win(board_copy, human):
                return i

    # 3. Take the center if available
    if board[4] == '5':
        return 4

    # 4. Take any remaining random spot
    available_moves = [i for i in range(9) if board[i] not in ['X', 'O']]
    return random.choice(available_moves)

def play_game():
    """Main game loop."""
    print("Welcome to Tic-Tac-Toe!")
    print("name:H BHOOMIKA, USN:!BM24CS107")
    print("You are 'X' and the computer is 'O'. Enter a number from 1-9 to make your move.")
    print_board()

    while True:
        # --- Human Turn ---
        while True:
            try:
                move = int(input("Your turn (1-9): ")) - 1
                if 0 <= move <= 8 and board[move] not in ['X', 'O']:
```

```
                    board[move] = human
                    break
                else:
                    print("That spot is already taken or invalid. Try again.")
            except ValueError:
                print("Invalid input. Please enter a number between 1 and 9.")

        print_board()
        if check_win(board, human):
            print("🎉 Congratulations! You beat the computer!")
            break
        if check_tie(board):
            print("🤝 It's a tie!")
            break

        # --- Computer Turn ---
        print("Computer is making a move...")
        comp_move = get_computer_move()
        board[comp_move] = computer

        print_board()
        if check_win(board, computer):
            print("💻 The computer wins! Better luck next time.")
            break
        if check_tie(board):
            print("🤝 It's a tie!")
            break

if __name__ == "__main__":
    play_game()
```

```
Welcome to Tic-Tac-Toe!
name:H BHOOMIKA, USN:!BM24CS107
You are 'X' and the computer is 'O'. Enter a number from 1-9 to make your move.


 1 | 2 | 3
---|---|---
 4 | 5 | 6
---|---|---
 7 | 8 | 9


Your turn (1-9): 5


 1 | 2 | 3
---|---|---
 4 | X | 6
---|---|---
 7 | 8 | 9


Computer is making a move...


 1 | 2 | 3
---|---|---
 4 | X | 6
---|---|---
 7 | O | 9


Your turn (1-9): 3


 1 | 2 | X
---|---|---
 4 | X | 6
---|---|---
 7 | O | 9


Computer is making a move...


 1 | 2 | X
---|---|---
 4 | X | 6
---|---|---
 O | O | 9
```

```
Your turn (1-9): 9


 1 | 2 | X
---|---|---
 4 | X | 6
```

```
# Vacuum Cleaner Agent - Simple Reflex Agent

def vacuum_cleaner():
    # Initial state of rooms
    room = {
        'A': 'Dirty',
        'B': 'Dirty'
    }

    # Starting position of the vacuum cleaner
    position = 'A'

    print("Initial State:")
    print(room)
    print("Vacuum Cleaner is at Room", position)

    # Continue until both rooms are clean
    while 'Dirty' in room.values():

        print("\nCurrent Position:", position)
        print("Room Status:", room)

        # If current room is dirty, clean it
        if room[position] == 'Dirty':
            print("Action: Suck")
            room[position] = 'Clean'

        # If current room is clean, move to the other room
        else:
            if position == 'A':
                position = 'B'
                print("Action: Move Right")
            else:
                position = 'A'
                print("Action: Move Left")

    print("\nFinal State:")
    print(room)
    print("Both rooms are clean.")
    print("Vacuum Cleaner has completed its task.")


# Call the function
vacuum_cleaner()
```

```
Initial State:
{'A': 'Dirty', 'B': 'Dirty'}
Vacuum Cleaner is at Room A

Current Position: A
Room Status: {'A': 'Dirty', 'B': 'Dirty'}
Action: Suck

Current Position: A
Room Status: {'A': 'Clean', 'B': 'Dirty'}
Action: Move Right

Current Position: B
Room Status: {'A': 'Clean', 'B': 'Dirty'}
Action: Suck

Final State:
{'A': 'Clean', 'B': 'Clean'}
Both rooms are clean.
Vacuum Cleaner has completed its task.
```